8. 합의 메커니즘 분석

개요

Reth는 Ethereum의 Proof of Stake (PoS) 합의 메커니즘을 구현하기 위해 `EthBeaconConsensus` 를 사용합니다. 이는 Ethereum 2.0의 Beacon Chain과 통합되어 작동하며, Engine API를 통해 합의 레이어(Consensus Layer)와 실행 레이어(Execution Layer) 간의 통신을 담당합니다.

Reth의 합의 시스템은 다음과 같은 핵심 특징을 가집니다:

- **Beacon Chain 통합**: Ethereum 2.0 Beacon Chain과의 완전한 호환성
- **Engine API 구현**: 합의 레이어와 실행 레이어 간의 표준화된 통신 인터페이스
- **모듈화된 설계**: 다양한 합의 메커니즘을 지원할 수 있는 확장 가능한 구조
- **검증 중심**: 블록과 헤더의 엄격한 검증을 통한 네트워크 보안 보장

핵심 구조체 및 트레이트

EthBeaconConsensus

`EthBeaconConsensus` 는 Ethereum Beacon Chain 합의를 구현하는 핵심 구조체입니다:

```
#[derive(Debug, Clone)]
pub struct EthBeaconConsensus<ChainSpec> {
/// 체인 스펙 설정
chain_spec: Arc<ChainSpec>,
}
```

```rust
impl<ChainSpec: EthChainSpec + EthereumHardforks> EthBeaconConsensus<ChainSpec>

/// 새로운 EthBeaconConsensus 인스턴스 생성

pub const fn new(chain_spec: Arc<ChainSpec>) -> Self {

Self { chain_spec }

}



/// 연관된 체인 스펙 반환

pub const fn chain_spec(&self) -> &Arc<ChainSpec> {

&self.chain_spec

}

}
```

Consensus 트레이트

합의 메커니즘의 핵심 인터페이스를 정의합니다:

```rust
pub trait Consensus<B: Block>: HeaderValidator<B::Header> {

type Error;



/// 바디 필드 값이 헤더와 일치하는지 확인

fn validate_body_against_header(

&self,

body: &B::Body,

header: &SealedHeader<B::Header>,
```

```
) -> Result<(), Self::Error>;


/// 실행 전 블록 검증 (월드 스테이트 무관)
fn validate_block_pre_execution(&self, block: &SealedBlock<B>) -> Result<(), Se
}
```

HeaderValidator 트레이트

헤더 검증을 담당하는 핵심 트레이트입니다:

```
pub trait HeaderValidator<H = Header>: Debug + Send + Sync {
/// 헤더가 올바르고 합의 사양을 따르는지 검증
fn validate_header(&self, header: &SealedHeader<H>) -> Result<(), ConsensusErro


/// 부모 헤더와의 관계가 올바른지 검증
fn validate_header_against_parent(
&self,
header: &SealedHeader<H>,
parent: &SealedHeader<H>,
) -> Result<(), ConsensusError>;
}
```

FullConsensus 트레이트

실행 결과를 포함한 완전한 합의 검증을 제공합니다:

```
pub trait FullConsensus<N: NodePrimitives>: Consensus<N::Block> {
/// 실행 후 블록 검증 (월드 스테이트 고려)
fn validate_block_post_execution(
&self,
block: &RecoveredBlock<N::Block>,
result: &BlockExecutionResult<N::Receipt>,
) -> Result<(), ConsensusError>;
}
```

주요 기능

1. 헤더 검증

`EthBeaconConsensus` 는 다음과 같은 헤더 검증을 수행합니다:

```
impl<H, ChainSpec> HeaderValidator<H> for EthBeaconConsensus<ChainSpec>
where
H: BlockHeader,
ChainSpec: EthChainSpec<Header = H> + EthereumHardforks + Debug + Send + Sync,
{
fn validate_header(&self, header: &SealedHeader<H>) -> Result<(), ConsensusErro
let header = header.header();
let is_post_merge = self.chain_spec.is_paris_active_at_block(header.number());
```

```
// The Merge 이후 검증
if  is_post_merge {
// 난이도는 0이어야 함
if !header.difficulty().is_zero() {
return Err(ConsensusError::TheMergeDifficultyIsNotZero);
}

// Nonce는 0이어야 함
if !header.nonce().is_some_and(|nonce| nonce.is_zero()) {
return Err(ConsensusError::TheMergeNonceIsNotZero);
}

// Ommer root는 비어있어야 함
if  header.ommers_hash() != EMPTY_OMMER_ROOT_HASH {
return Err(ConsensusError::TheMergeOmmerRootIsNotEmpty);
}
}

// 기본 헤더 검증
validate_header_extra_data(header)?;
validate_header_gas(header)?;
validate_header_base_fee(header, &self.chain_spec)?;
```

```
// EIP-4895: Shanghai 이후 withdrawals 검증
if  self.chain_spec.is_shanghai_active_at_timestamp(header.timestamp()) &&
header.withdrawals_root().is_none()
{
return Err(ConsensusError::WithdrawalsRootMissing)
}

// EIP-4844: Cancun 이후 blob 필드 검증
if  self.chain_spec.is_cancun_active_at_timestamp(header.timestamp()) {
validate_4844_header_standalone(
header,
self.chain_spec
.blob_params_at_timestamp(header.timestamp())
.unwrap_or_else(BlobParams::cancun),
)?;
}

Ok(())
}
}
```

2. 부모-자식 헤더 관계 검증

```
fn  validate_header_against_parent(
&self,
header: &SealedHeader<H>,
parent: &SealedHeader<H>,
) -> Result<(), ConsensusError> {
// 해시와 블록 번호 검증
validate_against_parent_hash_number(header.header(), parent)?;


// 타임스탬프 검증
validate_against_parent_timestamp(header.header(), parent.header())?;


// 가스 한도 검증
validate_against_parent_gas_limit(header, parent, &self.chain_spec)?;


// EIP-1559 base fee 검증
validate_against_parent_eip1559_base_fee(
header.header(),
parent.header(),
&self.chain_spec,
)?;


// EIP-4844 blob 가스 필드 검증
```

```
if  let Some(blob_params) = self.chain_spec.blob_params_at_timestamp(header.time
validate_against_parent_4844(header.header(), parent.header(), blob_params)?;
}


Ok(())
}
```

3. 실행 후 블록 검증

```
impl<ChainSpec, N> FullConsensus<N> for EthBeaconConsensus<ChainSpec>
where
ChainSpec: Send + Sync + EthChainSpec<Header = N::BlockHeader> + EthereumHardfor
N: NodePrimitives,
{
fn  validate_block_post_execution(
&self,
block: &RecoveredBlock<N::Block>,
result: &BlockExecutionResult<N::Receipt>,
) -> Result<(), ConsensusError> {
validate_block_post_execution(
block,
&self.chain_spec,
&result.receipts,
```

```
&result.requests
)
}
}
```

Engine API를 통한 합의 레이어 통신

Engine API 구조

Engine API는 합의 레이어와 실행 레이어 간의 통신을 담당하는 핵심 인터페이스입니다:

```
pub struct EngineApi<Provider, PayloadT: PayloadTypes, Pool, Validator, ChainSp
inner: Arc<EngineApiInner<Provider, PayloadT, Pool, Validator, ChainSpec>>,
}
```

주요 Engine API 메서드

1. New Payload 처리

```
/// engine_newPayloadV1 - Payload 검증 및 처리
pub async fn new_payload_v1(
&self,
payload: PayloadT::ExecutionData,
) -> EngineApiResult<PayloadStatus> {
let payload_or_attrs = PayloadOrAttributes::from_execution_payload(&payload);
```

```
// 버전별 필드 검증
self.inner
.validator
.validate_version_specific_fields(EngineApiMessageVersion::V1, payload_or_attrs)

// 합의 엔진에 payload 전달
Ok(self
.inner
.beacon_consensus
.new_payload(payload)
.await
.inspect(|_| self.inner.on_new_payload_response())?)
}
```

2. Fork Choice Update 처리

```
/// engine_forkchoiceUpdatedV1 - 포크 선택 업데이트
pub  async  fn  fork_choice_updated_v1(
&self,
state: ForkchoiceState,
payload_attrs: Option<EngineT::PayloadAttributes>,
) -> EngineApiResult<ForkchoiceUpdated> {
```

```
self.validate_and_execute_forkchoice(
EngineApiMessageVersion::V1,
state,
payload_attrs
).await
}
```

3. Payload 조회

```
/// engine_getPayloadV1 - 빌드된 payload 조회
pub  async  fn  get_payload_v1(
&self,
payload_id: PayloadId,
) -> EngineApiResult<EngineT::ExecutionPayloadEnvelopeV1> {
self.get_built_payload(payload_id)
.await?
.try_into()
.map_err(|_| EngineApiError::UnknownPayload)
}
```

Engine API 트레이트 정의

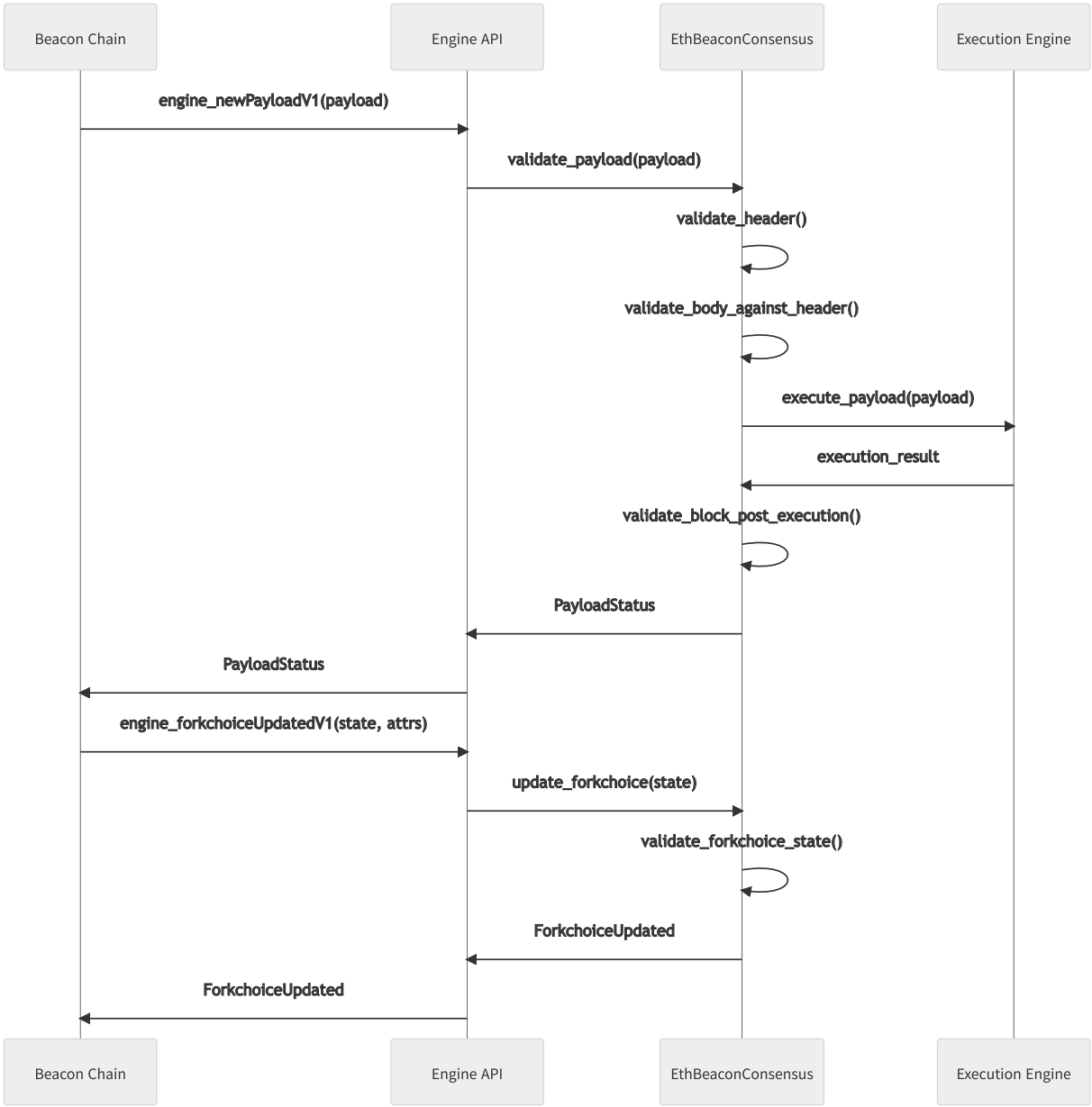
```
#[rpc(server, namespace = "engine")]
pub  trait EngineApi<Engine: EngineTypes> {
```

```
/// 새로운 payload 처리
#[method(name = "newPayloadV1")]
async fn new_payload_v1(&self, payload: ExecutionPayloadV1) -> RpcResult<Paylo


/// 포크 선택 업데이트
#[method(name = "forkchoiceUpdatedV1")]
async fn fork_choice_updated_v1(
&self,
fork_choice_state: ForkchoiceState,
payload_attributes: Option<Engine::PayloadAttributes>,
) -> RpcResult<ForkchoiceUpdated>;


/// Payload 조회
#[method(name = "getPayloadV1")]
async fn get_payload_v1(
&self,
payload_id: PayloadId,
) -> RpcResult<Engine::ExecutionPayloadEnvelopeV1>;
}
```

Proof of Stake 통합 방식

1. Beacon Chain과의 상호작용

Reth는 다음과 같은 방식으로 Beacon Chain과 통합됩니다:


Beacon Chain
Engine API
EthBeaconConsensus
Execution Engine
engine_newPayloadV1(payload)
validate_payload(payload)
validate_header()
validate_body_against_header()
execute_payload(payload)
execution_result
validate_block_post_execution()
PayloadStatus
PayloadStatus
engine_forkchoiceUpdatedV1(state, attrs)
update_forkchoice(state)
validate_forkchoice_state()
ForkchoiceUpdated
ForkchoiceUpdated
Beacon Chain
Engine API
EthBeaconConsensus
Execution Engine

2. 합의 검증 프로세스

```
// 1. 헤더 검증
consensus.validate_header(&sealed_header)?;

// 2. 부모와의 관계 검증
consensus.validate_header_against_parent(&header, &parent)?;

// 3. 실행 전 블록 검증
consensus.validate_block_pre_execution(&sealed_block)?;

// 4. 블록 실행
let execution_result = executor.execute_block(&block)?;

// 5. 실행 후 검증
consensus.validate_block_post_execution(&block, &execution_result)?;
```

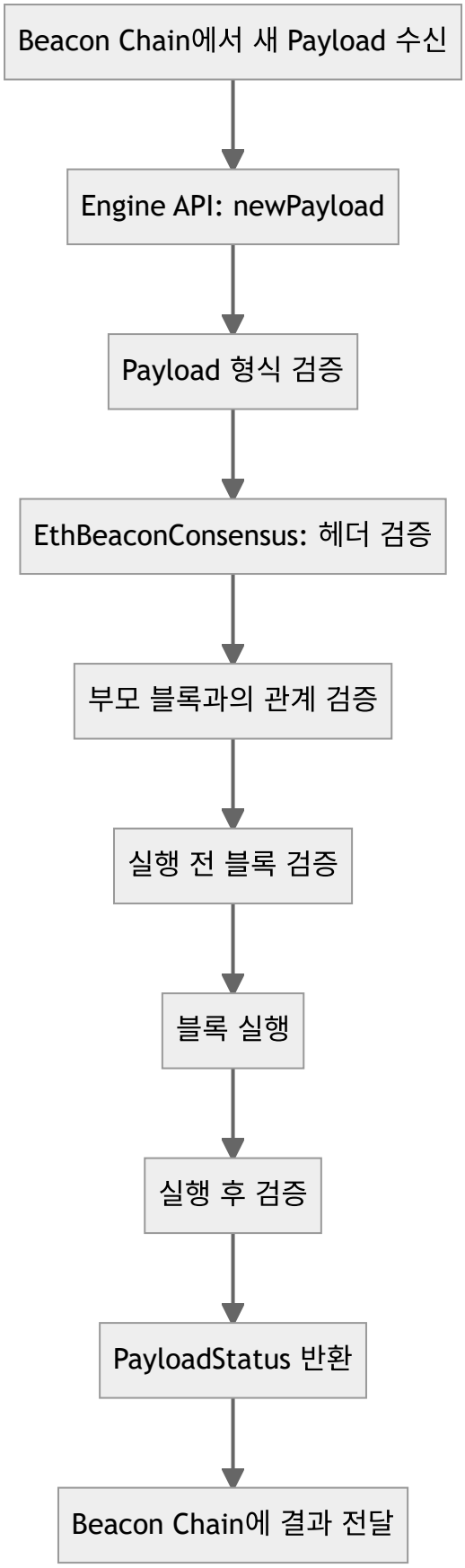
3. 포크 선택 규칙

```
impl ForkchoiceStateTracker {
/// 새로운 포크 선택 상태 업데이트
pub fn update_forkchoice_state(&mut self, state: ForkchoiceState) {
```

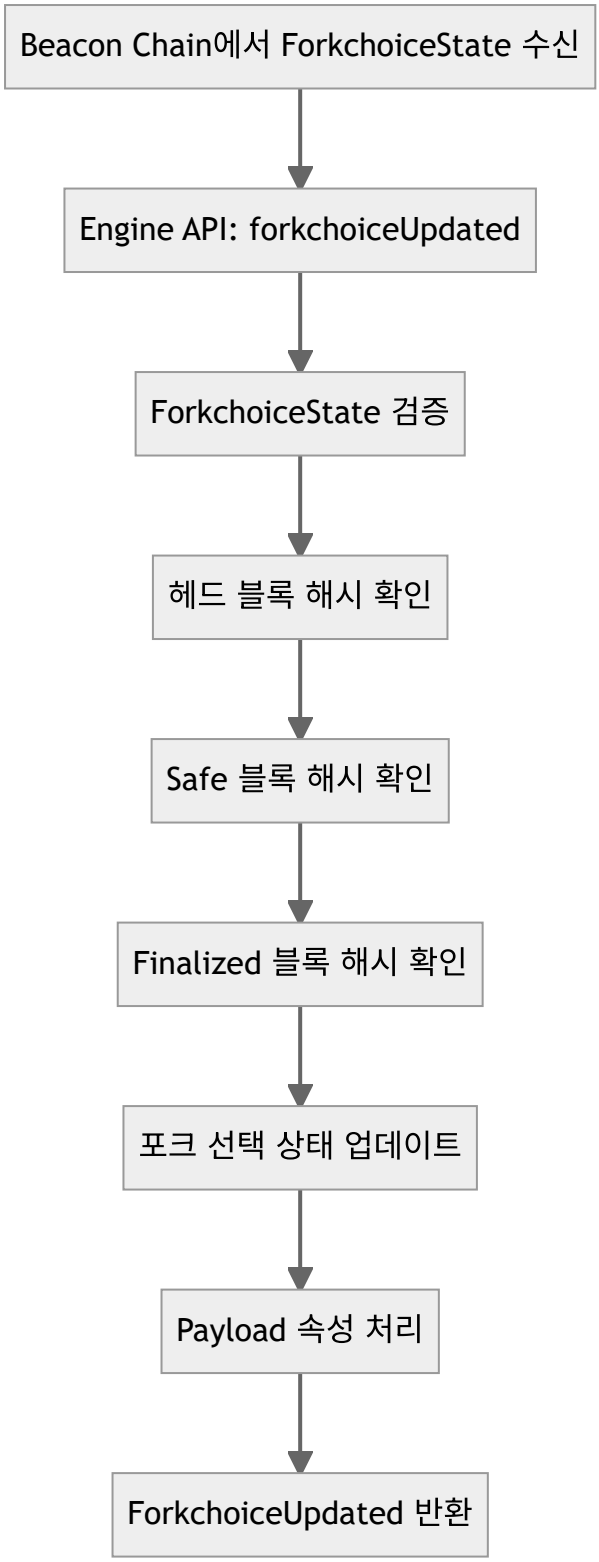
```
self.head_block_hash = state.head_block_hash;

self.safe_block_hash = state.safe_block_hash;

self.finalized_block_hash = state.finalized_block_hash;

}



/// 현재 포크 선택 상태 반환

pub fn forkchoice_state(&self) -> ForkchoiceState {

ForkchoiceState {

head_block_hash: self.head_block_hash,

safe_block_hash: self.safe_block_hash,

finalized_block_hash: self.finalized_block_hash,

}

}

}
```

프로세스 플로우

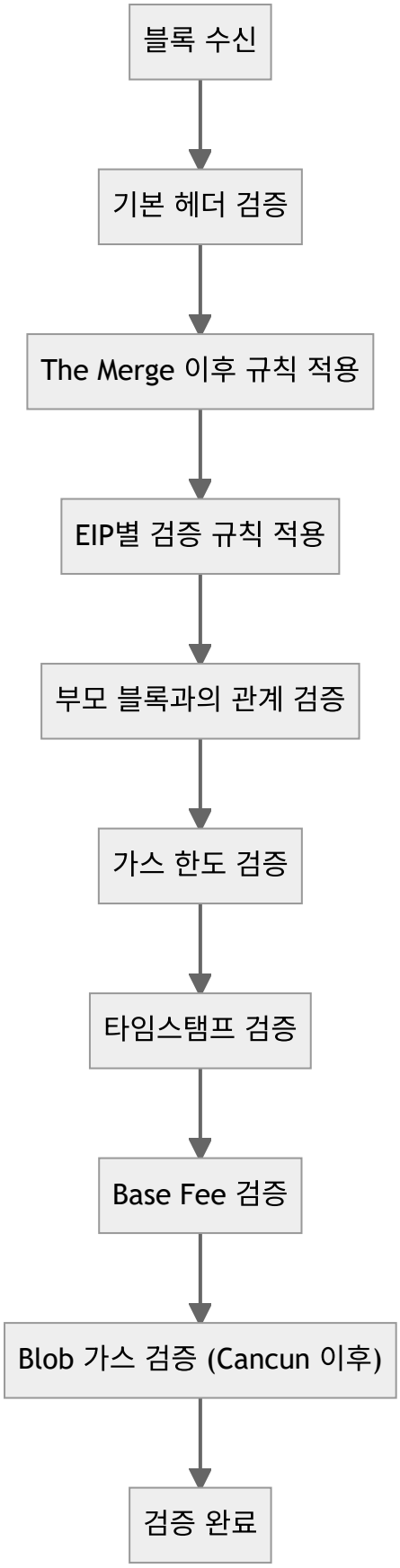
1. 새로운 블록 처리 플로우



2. 포크 선택 업데이트 플로우


Beacon Chain에서 ForkchoiceState 수신
Engine API: forkchoiceUpdated
ForkchoiceState 검증
헤드 블록 해시 확인
Safe 블록 해시 확인
Finalized 블록 해시 확인
포크 선택 상태 업데이트
Payload 속성 처리
ForkchoiceUpdated 반환

3. 합의 검증 단계



설정 및 커스터마이징

1. 합의 엔진 설정

```
// 노드 빌더에서 합의 엔진 설정

impl<Node> ConsensusBuilder<Node> for EthereumConsensusBuilder

where

Node: FullNodeTypes<Types: NodeTypes<ChainSpec = ChainSpec>>,

{

type Consensus = Arc<EthBeaconConsensus<ChainSpec>>;



async  fn  build_consensus(self, ctx: &BuilderContext<Node>) -> eyre::Result<Sel

Ok(Arc::new(EthBeaconConsensus::new(ctx.chain_spec())))

}

}
```

2. Engine API 설정

```
// Engine API 인스턴스 생성

let  engine_api = EngineApi::new(

provider,

chain_spec,

beacon_consensus,

payload_store,

tx_pool,

task_spawner,
```

```
client_version,
capabilities,
validator,
accept_execution_requests_hash,
);
```

3. 체인별 설정

```
// 메인넷 설정
let mainnet_consensus = EthBeaconConsensus::new(Arc::new(MAINNET.clone()));

// 테스트넷 설정
let goerli_consensus = EthBeaconConsensus::new(Arc::new(GOERLI.clone()));

// 커스텀 체인 설정
let custom_spec = ChainSpecBuilder::default()
.chain(Chain::from_id(12345))
.genesis(genesis_config)
.paris_activated()
.shanghai_activated()
.build();
let custom_consensus = EthBeaconConsensus::new(Arc::new(custom_spec));
```

관련 컴포넌트

1. 의존성 관계

- **상위 의존성**: Engine API, Node Builder
- **하위 의존성**: Chain Spec, Execution Engine, Storage Provider
- **피어 의존성**: Payload Builder, Transaction Pool

2. 상호작용 방식

```
// Engine API와의 상호작용
engine_api.new_payload_v1(payload).await?;
engine_api.fork_choice_updated_v1(state, attrs).await?;

// 실행 엔진과의 상호작용
let execution_result = executor.execute_block(&block)?;
consensus.validate_block_post_execution(&block, &execution_result)?;

// 스토리지와의 상호작용
let parent_header = provider.header(&parent_hash)?;
consensus.validate_header_against_parent(&header, &parent_header)?;
```

3. 이벤트 및 메트릭

```
// 합의 관련 메트릭
```

```rust
pub struct ConsensusMetrics {
pub header_validations: Counter,
pub block_validations: Counter,
pub validation_errors: Counter,
pub forkchoice_updates: Counter,
}

// 합의 이벤트
pub enum ConsensusEvent {
HeaderValidated { hash: BlockHash },
BlockValidated { hash: BlockHash, number: BlockNumber },
ForkchoiceUpdated { head: BlockHash, safe: BlockHash, finalized: BlockHash },
ValidationError { error: ConsensusError },
}
```

Reth의 합의 메커니즘은 Ethereum의 Proof of Stake 시스템과 완전히 호환되며, Engine API를 통해 Beacon Chain과 안전하고 효율적으로 통신합니다. 모듈화된 설계를 통해 다양한 네트워크 요구사항에 맞게 확장 가능하며, 엄격한 검증 프로세스를 통해 네트워크의 보안과 무결성을 보장합니다.